

Corporación Universitaria Minuto de Dios UNIMINUTO



UNIMINUTO
Corporación Universitaria Minuto de Dios
Educación de calidad al alcance de todos

FACULTAD DE INGENIERÍA

SEGURIDAD EN ENTORNOS CLOUD Y DEVOPS NRC-11-75062

Trabajo Final en Ciberseguridad con Énfasis en DevSecOps

PRESENTA:

**CARLOS ALBERTO GONZALEZ BRAVO
DIEGO ARMANDO HERNANDEZ URDANETA
ROBERTO CARLOS MUÑOZ NOGUERA
DANIEL MAURICIO DAZA BORJA**

PRESENTADO A:

JAIDER OSPINA NAVAS

MAYO 2026

Tabla de Contenido

1. INTRODUCCIÓN4

1.1 Contexto y Justificación..... 4

1.2 Descripción de la Aplicación Desarrollada..... 4

2. OBJETIVOS.....5

2.1 Objetivo General..... 5

2.2 Objetivos Específicos..... 5

3. ALCANCE Y LIMITACIONES.....6

4. ESTRUCTURA DEL DOCUMENTO7

5. ARQUITECTURA.....7

5.1 Diagrama de Componentes 7

5.2 Diagrama de Despliegue 8

5.3 Diagrama de Secuencia — Flujo de Autenticación 8

5.4 Diagrama de Secuencia — Creación de Nota..... 9

5.5 Diagrama de Casos de Uso 9

5.6 Descripción de Servicios..... 10

5.7 Comunicación entre Servicios 10

5.8 Capas de Seguridad..... 10

6. MODELADO DE AMENAZAS10

6.1 DFD Nivel 1 — Flujo Interno de Datos..... 11

6.2 Análisis STRIDE Detallado..... 11

6.3 Matriz de Riesgo 13

7. IMPLEMENTACIÓN DEL PIPELINE.....14

7.1 Visión General 14

7.2 Fase Code — Análisis Estático..... 14

7.3 Fase Dependencies — SCA..... 15

7.4 Fase Build — Escaneo de Imágenes..... 15

7.5 Tests implementados:..... 16

7.6 Fase DAST — OWASP ZAP 16

7.7 Fase Release — Docker Hub 16

7.8 Escaneo de IaC — Checkov 16

8. RESULTADOS DE SEGURIDAD17

8.1	Vulnerabilidades Encontradas y Remediadas	17
8.2	Controles de Seguridad Implementados	17
8.3	Excepciones Documentadas.....	18
9.	MONITOREO Y OBSERVABILIDAD	18
9.1	Observabilidad Implementada	18
9.2	Arquitectura de Monitoreo Planificada.....	19
9.3	Métricas Prioritarias Identificadas	19
10.	CONCLUSIONES	21
10.1	Desafíos Encontrados.....	21
10.2	Limitaciones.....	21
10.3	Lecciones Aprendidas	21
10.4	Trabajo Futuro	21

SEGURIDAD EN ENTORNOS CLOUD Y DEVOPS NRC-11-75062

Trabajo Final en Ciberseguridad con Énfasis en DevSecOps

1. INTRODUCCIÓN

1.1 Contexto y Justificación

La aceleración de los ciclos de desarrollo de software, impulsada por la adopción masiva de metodologías ágiles y prácticas de integración y entrega continua (CI/CD), ha generado una tensión estructural en la industria tecnológica: la velocidad de entrega tiende a desplazar la seguridad hacia los márgenes del proceso productivo, convirtiéndola en una actividad reactiva que se ejecuta —cuando se ejecuta— al final del ciclo. Esta práctica, conocida coloquialmente como **security as an afterthought**, ha demostrado ser costosa tanto en términos económicos como reputacionales, tal como evidencian los informes anuales de brechas de seguridad publicados por organizaciones como CISA, NIST y OWASP.

Frente a esta problemática, el paradigma **DevSecOps** propone una reconfiguración fundamental del rol de la seguridad en el ciclo de vida del software: en lugar de constituir una fase terminal de revisión, la seguridad se integra como una práctica continua, automatizada y trazable en cada etapa, desde la planificación inicial hasta la operación en producción. Este enfoque no implica únicamente la incorporación de herramientas de escaneo en el pipeline; supone, ante todo, un cambio cultural que exige que los equipos de desarrollo asuman corresponsabilidad en la postura de seguridad del sistema que construyen.

El presente trabajo tiene como propósito central la demostración práctica de este paradigma. Para ello, se ha diseñado, implementado y documentado un pipeline DevSecOps de ciclo completo sobre una aplicación contenerizada de arquitectura de microservicios, integrando herramientas de seguridad de código abierto en cada una de sus fases. En consonancia con los lineamientos del programa, la aplicación desarrollada constituye el vehículo que permite ejercitar el pipeline; es este último el objeto de evaluación y el producto intelectual central del trabajo.

1.2 Descripción de la Aplicación Desarrollada

La aplicación seleccionada para este trabajo es **Secure Workspace**, una plataforma web de gestión segura de notas y espacios de trabajo personales. Conceptualmente afín a herramientas como Notion o Obsidian, Secure Workspace prioriza la privacidad del usuario y la seguridad de sus datos como atributos de diseño de primera clase, no como características adicionales.

La elección de esta aplicación responde a criterios de conveniencia técnica y pedagógica:

Pertinencia del dominio de seguridad. Al gestionar información personal de usuarios — credenciales, notas de contenido potencialmente sensible y tokens de sesión—, la aplicación presenta una superficie de ataque representativa que justifica la aplicación rigurosa del modelo STRIDE y el conjunto completo de controles DevSecOps exigidos.

Suficiencia arquitectónica. El sistema implementa una arquitectura de microservicios compuesta por cinco componentes independientes: una interfaz de usuario SPA desarrollada en React, un API Gateway implementado en Python con FastAPI, un worker de procesamiento asíncrono basado en Celery, una base de datos relacional PostgreSQL y un broker de mensajes Redis. Esta arquitectura introduce los patrones de comunicación, la gestión de secretos y las dependencias de terceros necesarios para ejercitar la totalidad del pipeline de seguridad.

Representatividad de vectores de amenaza. La implementación de autenticación con JSON Web Tokens (JWT), autorización basada en roles, operaciones CRUD sobre datos persistentes y comunicación asíncrona entre servicios reproduce los vectores de riesgo más prevalentes en sistemas web modernos, según la clasificación OWASP Top 10, dotando de significado real al análisis de amenazas.

Alineación con el software libre. La totalidad del stack tecnológico —React, FastAPI, Celery, PostgreSQL, Redis y Docker— se basa en software de código abierto con licencias permisivas, en correspondencia con los requisitos del programa y con el espíritu de transparencia y reproducibilidad que caracteriza a las buenas prácticas DevSecOps.

2. OBJETIVOS

2.1 Objetivo General

Diseñar, implementar y documentar un pipeline DevSecOps de ciclo completo para la aplicación contenerizada Secure Workspace, integrando herramientas de seguridad de código abierto en cada fase del ciclo de vida del desarrollo de software, con el propósito de demostrar la viabilidad y el valor operativo del paradigma DevSecOps en un proyecto de software real.

2.2 Objetivos Específicos

1. Elaborar el modelado de amenazas del sistema mediante la metodología STRIDE y la herramienta OWASP Threat Dragon, generando Diagramas de Flujo de Datos (DFD) de nivel 0 y nivel 1 que fundamenten las decisiones de diseño seguro.
2. Automatizar el análisis estático de seguridad del código fuente (SAST) mediante las herramientas Gitleaks, Bandit y Semgrep, con el objetivo de detectar secretos expuestos, vulnerabilidades en código Python y patrones de programación inseguros catalogados en el OWASP Top 10.

3. Implementar el análisis de composición de software (SCA) sobre los archivos de dependencias de cada microservicio mediante Trivy, configurando el pipeline para interrumpir la ejecución ante la detección de CVEs de severidad CRITICAL o HIGH, salvo excepción debidamente documentada y justificada.
4. Integrar el escaneo de infraestructura como código (IaC) mediante Checkov, validando la configuración de Dockerfiles y manifiestos de Docker Compose contra controles de seguridad de referencia de la industria.
5. Ejecutar análisis dinámico de seguridad (DAST) con OWASP ZAP sobre un entorno de *staging* efímero levantado en el propio pipeline, identificando vulnerabilidades en la API REST en condiciones de ejecución real.
6. Automatizar el proceso de publicación de imágenes Docker en Docker Hub con versionado semántico, condicionando la publicación al paso exitoso de la totalidad de las etapas de seguridad precedentes.
7. Implementar la infraestructura de despliegue mediante el enfoque de Infraestructura como Código (IaC) con Ansible, permitiendo el aprovisionamiento automatizado y reproducible del entorno de producción simulado.
8. Documentar de forma completa y reproducible cada decisión de diseño, resultado de herramienta y proceso de remediación de vulnerabilidades, de modo que el repositorio constituya por sí mismo una referencia técnica autosuficiente.

3. ALCANCE Y LIMITACIONES

El alcance del presente trabajo comprende el ciclo DevSecOps completo desde la fase de planificación y modelado de amenazas hasta el despliegue automatizado mediante Ansible y la publicación versionada de imágenes en Docker Hub. El entorno de producción simulado opera sobre Docker Compose en infraestructura local, decisión deliberada que garantiza la reproducibilidad del proyecto en cualquier máquina con Docker instalado, sin dependencia de proveedores de nube pública ni costos asociados.

Se identifican las siguientes limitaciones, asumidas como riesgo residual aceptado y documentadas en consecuencia:

- Orquestación de contenedores avanzada: La implementación no incluye despliegue sobre Kubernetes o K3s. La orquestación se realiza mediante Docker Compose, considerada suficiente para el alcance de un entorno de producción simulado.
- Monitoreo y observabilidad: La configuración de un stack de observabilidad con Prometheus, Grafana y Loki queda identificada como línea de trabajo futuro, al exceder el alcance temporal del proyecto.
- Terraform: La gestión de infraestructura en nube mediante Terraform no se implementa en este proyecto; la IaC se limita al uso de Ansible para el aprovisionamiento del entorno local.

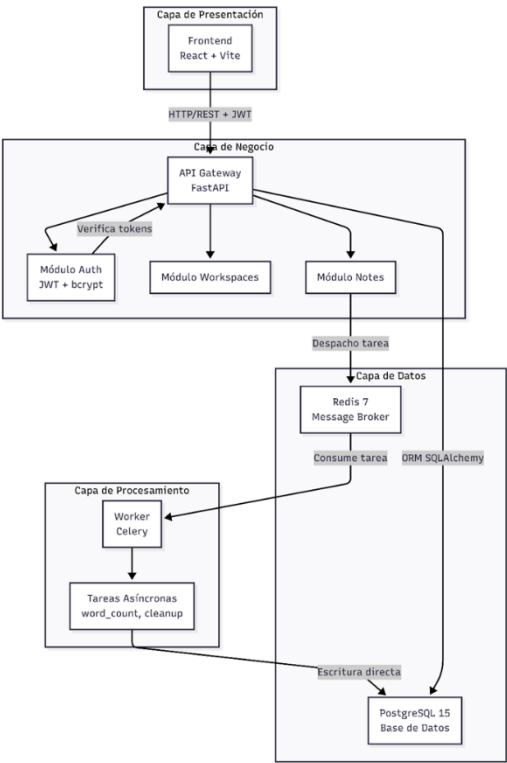
4. ESTRUCTURA DEL DOCUMENTO

El presente informe se organiza en las secciones que se detallan a continuación, siguiendo la estructura definida por los lineamientos indicados:

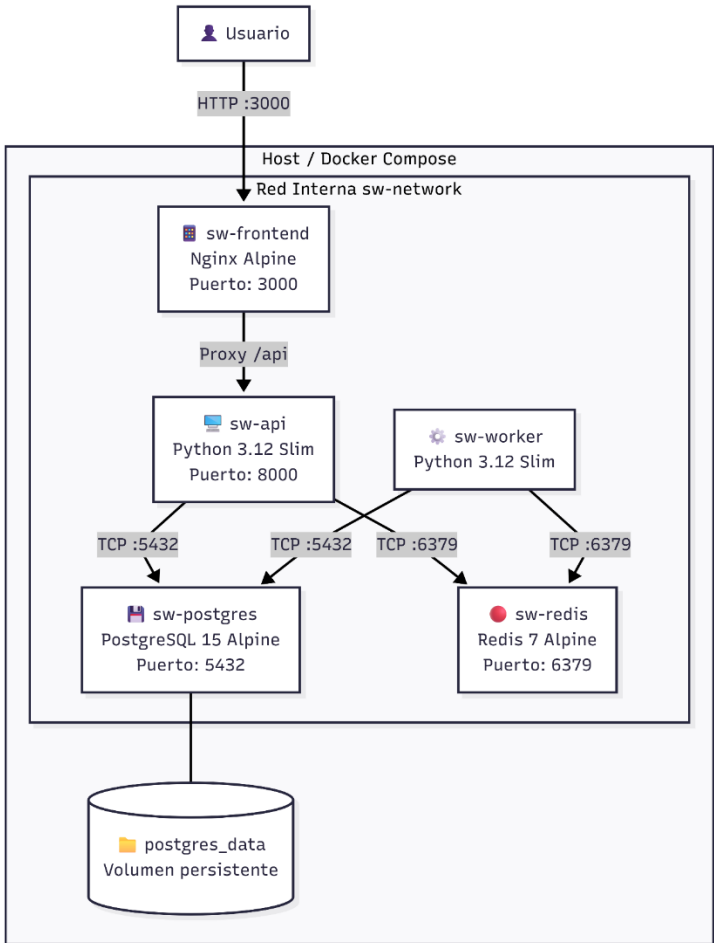
5. Arquitectura	Descripción de la arquitectura de microservicios, decisiones de diseño y diagramas UML: componentes, despliegue, secuencia y casos de uso
6. Modelado de Amenazas	Resultados del modelado con OWASP Threat Dragon, DFD nivel 0 y nivel 1, análisis STRIDE por categoría y contramedidas implementadas
7. Implementación del Pipeline	Descripción detallada de cada fase del pipeline CI/CD, herramientas integradas, políticas de bloqueo y fragmentos del archivo de workflow
8. Resultados de Seguridad	Reportes representativos de cada herramienta, tabla consolidada de hallazgos con severidad, estado y justificación
9. Monitoreo y Observabilidad	Estado de implementación e identificación como trabajo futuro
10. Conclusiones	Desafíos encontrados, limitaciones, lecciones aprendidas y líneas de trabajo futuro

5. ARQUITECTURA

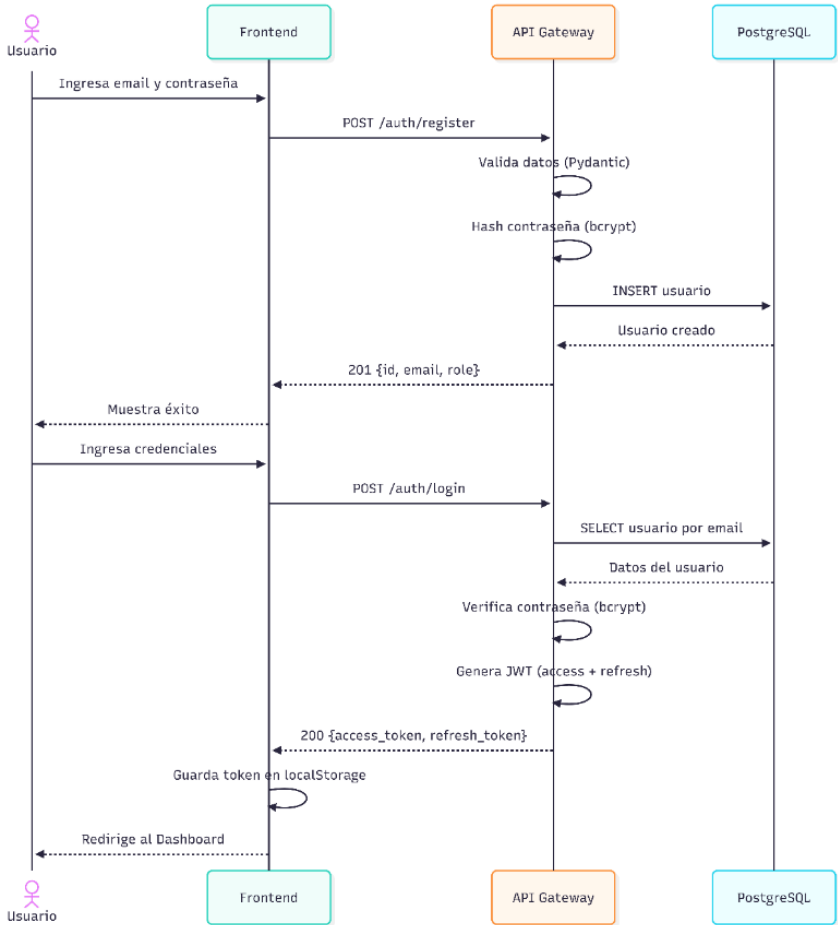
5.1 Diagrama de Componentes



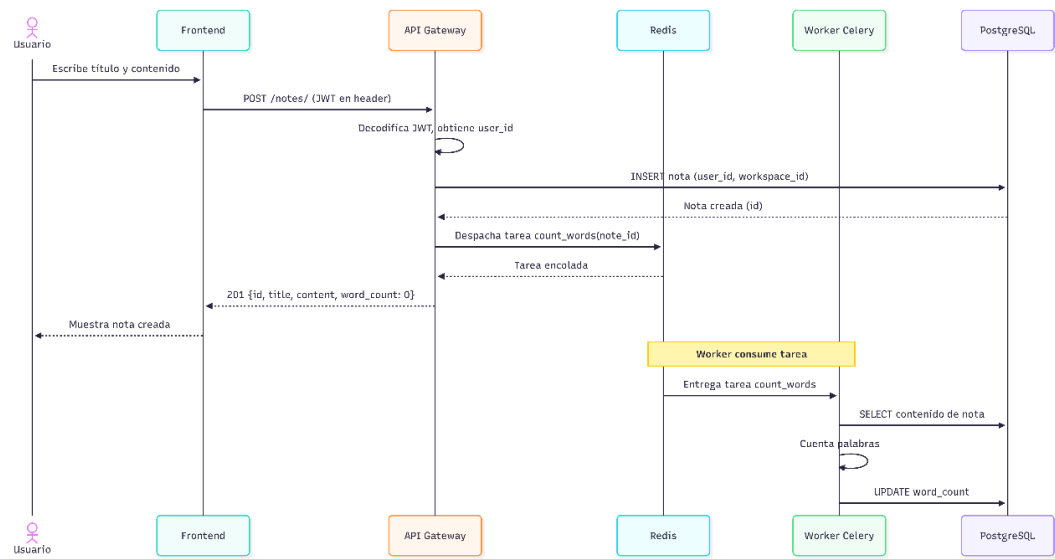
5.2 Diagrama de Despliegue



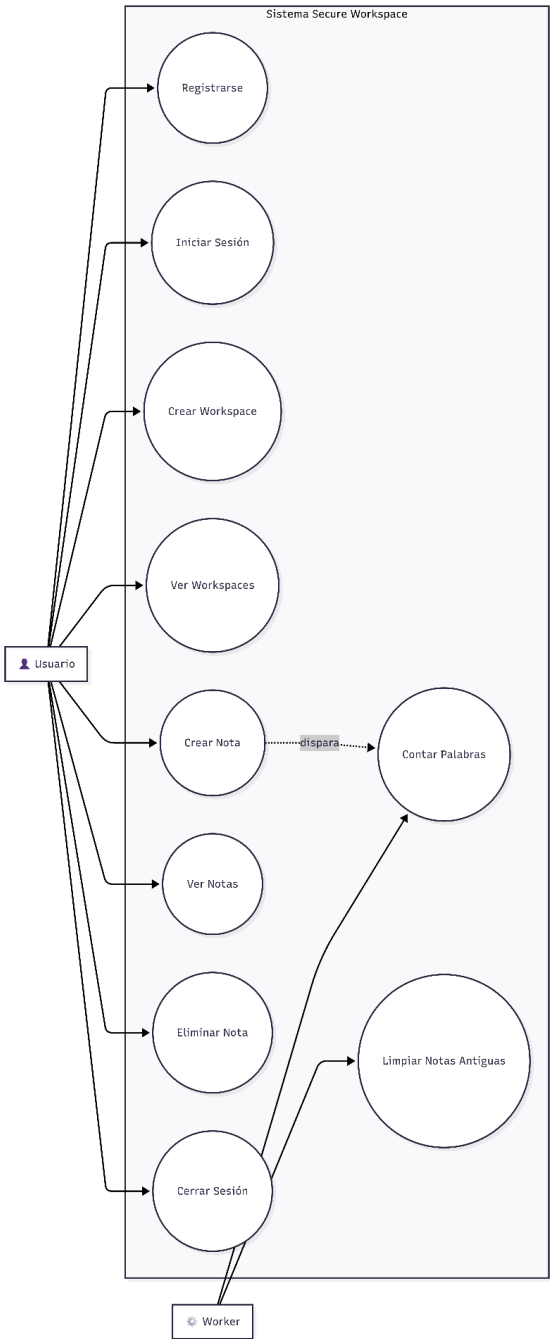
5.3 Diagrama de Secuencia — Flujo de Autenticación



5.4 Diagrama de Secuencia — Creación de Nota



5.5 Diagrama de Casos de Uso



5.6 Descripción de Servicios

Servicio	Tecnología	Puerto	Propósito
Frontend	React + Vite + Nginx	3000	Interfaz de usuario (login, dashboard, notas)
API Gateway	Python FastAPI	8000	API REST, autenticación, lógica de negocio
Worker	Python Celery	—	Tareas asíncronas (conteo de palabras, limpieza)
PostgreSQL	PostgreSQL 15 Alpine	5432	Almacenamiento persistente de datos
Redis	Redis 7 Alpine	6379	Broker de mensajes para Celery

5.7 Comunicación entre Servicios

- **Frontend → API Gateway:** HTTP/REST con tokens JWT Bearer.
- **API Gateway → Worker:** Despacho de tareas Celery vía broker Redis.
- **Worker → Base de Datos:** Conexión directa SQLAlchemy para escritura de metadatos.

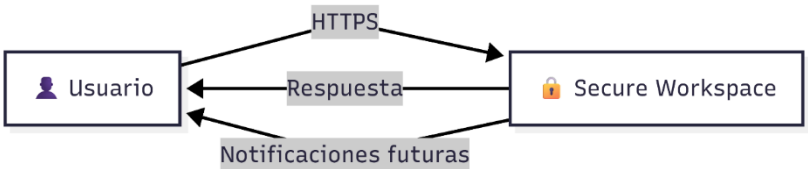
5.8 Capas de Seguridad

1. **Autenticación:** Tokens JWT de acceso (30 min) + refresco (7 días) con hash bcrypt.
2. **Autorización:** Control de acceso basado en roles (user / admin).
3. **Validación de Entradas:** Esquemas Pydantic en cada endpoint.
4. **Protección IDOR:** Todas las consultas limitadas al current_user.id.
5. **Gestión de Secretos:** Variables de entorno, nunca hardcodeados.
6. **Seguridad de Contenedores:** Usuarios no-root, imágenes base ligeras, Trivy.
7. **Escaneo IaC:** Checkov valida Dockerfiles y docker-compose.yml.

6. MODELADO DE AMENAZAS

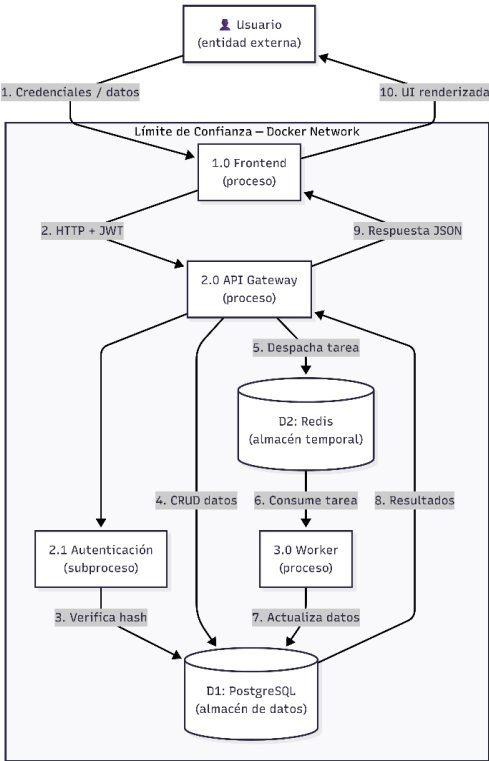
Metodología

Este modelo de amenazas sigue la metodología **STRIDE** de Microsoft y utiliza diagramas de flujo de datos (DFD) para identificar puntos de entrada y activos en riesgo.



El DFD nivel 0 muestra el sistema como una caja negra que recibe peticiones HTTP del usuario y devuelve respuestas con datos del workspace/notas.

6.1 DFD Nivel 1 — Flujo Interno de Datos



Activos Críticos

Activo	Sensibilidad	Ubicación	Protección
Credenciales de usuario	● ALTA	PostgreSQL (hash bcrypt)	bcrypt + salt, nunca en texto plano
Tokens JWT	● ALTA	Cliente (localStorage)	Expiración 30 min, firmados con HS256
Contenido de notas	● MEDIA	PostgreSQL	IDOR protegido, solo dueño accede
Secretos del entorno	● ALTA	.env, GitHub Secrets	.gitignore, Gitleaks en CI
Imágenes Docker	● MEDIA	Docker Hub	Trivy escanea CVEs, usuario no-root

6.2 Análisis STRIDE Detallado

Suplantación (Spoofing)

Amenaza	Probabilidad	Impacto	Mitigación
Atacante falsifica JWT	Media	Alto	JWT firmado con HS256, expiración 30 min
Reutilización de token robado	Media	Alto	Expiración corta, refresh token separado

Amenaza	Probabilidad	Impacto	Mitigación
Registro con email falso	Alta	Bajo	Validación de formato de email (Pydantic)

Manipulación (Tampering)

Amenaza	Probabilidad	Impacto	Mitigación
Inyección SQL	Baja	Crítico	ORM SQLAlchemy (consultas parametrizadas)
XSS en contenido de notas	Media	Medio	React escapa HTML por defecto
Modificar JWT en tránsito	Baja	Alto	Firma HS256, HTTPS en producción

Repudio (Repudiation)

Amenaza	Probabilidad	Impacto	Mitigación
Usuario niega acciones	Media	Medio	Timestamps en BD, logs con user_id
Borrado sin rastro	Baja	Medio	Campo created_at/updated_at en modelos

Divulgación de Información (Information Disclosure)

Amenaza	Probabilidad	Impacto	Mitigación
Secretos en código fuente	Media	Crítico	Gitleaks en CI, .gitignore para .env
IDOR — acceder notas de otro	Media	Alto	Filtro user_id en todas las queries
Información sensible en logs	Baja	Medio	No se loguean contraseñas ni tokens
CVEs en dependencias	Alta	Alto	Trivy SCA con exit-code: 1

Denegación de Servicio (Denial of Service)

Amenaza	Probabilidad	Impacto	Mitigación
Spam de registros	Alta	Medio	Rate limiting recomendado (futuro)
Agotamiento de conexiones BD	Baja	Alto	Connection pooling (SQLAlchemy)

Amenaza	Probabilidad	Impacto	Mitigación
Request flood	Media	Alto	Nginx como reverse proxy, healthchecks

Elevación de Privilegios (Elevation of Privilege)

Amenaza	Probabilidad	Impacto	Mitigación
Usuario accede endpoints admin	Media	Crítico	Decorador require_role en FastAPI
Escape de contenedor	Baja	Crítico	Usuario no-root, imágenes slim, red aislada
Modificar variables de entorno	Baja	Alto	Secretos en GitHub, no en código

Superficie de Ataque

Punto de Entrada	Protocolo	Auth Requerida	Riesgo	Mitigación
POST /auth/register	HTTP	No	Spam de cuentas	Validación de email, rate limiting
POST /auth/login	HTTP	No	Fuerza bruta	Expiración de tokens, bcrypt lento
GET/POST/workspaces	HTTP	JWT	IDOR	Filtro por user_id
GET/POST/DELETE /notes	HTTP	JWT	IDOR, DoS	Filtro por user_id, validación
Redis	TCP	No (red interna)	Acceso no autorizado	Red Docker aislada
PostgreSQL	TCP	Contraseña	Data breach	Red Docker aislada, credenciales en .env

6.3 Matriz de Riesgo

	Bajo	Medio	Alto	Crítico	
Alta Media Baja	Email	DoS reg XSS Logs	IDOR BD pool	Iny.SQL Escape contdor	

7. IMPLEMENTACIÓN DEL PIPELINE

7.1 Visión General

El pipeline DevSecOps se implementa en GitHub Actions mediante un único workflow (devsecops.yml) que automatiza 7 fases con herramientas FOSS.

Fase	Herramienta	Propósito	Política de fallo
Código (SAST)	Gitleaks	Detectar secretos filtrados	Reporta
Código (SAST)	Bandit	Vulnerabilidades en Python	Reporta
Código (SAST)	Semgrep	Patrones OWASP Top 10	Reporta
Dependencias (SCA)	Trivy	CVEs en dependencias	Falla con CRITICAL/HIGH
IaC	Checkov	Misconfiguración Docker/Compose	Falla con checks no aprobados
Build	Trivy	CVEs en imágenes Docker	Falla con CRITICAL
Test	Pytest + Coverage	Pruebas unitarias + cobertura	Falla si hay tests rotos

7.2 Fase Code — Análisis Estático

Gitleaks escanea el historial completo de Git buscando secretos:

```
- name: "Gitleaks - Detección de secretos"
  uses: gitleaks/gitleaks-action@v2
  continue-on-error:
    true
  env:
    GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
```

Bandit analiza el código Python del API Gateway y Worker:

```
- name: "Bandit - Análisis de seguridad Python"
  run: |
    pip install bandit
    bandit -r api-gateway/app/ -f json -o bandit-report.json || true
```

Semgrep busca patrones inseguros según reglas OWASP:

```
- name: "Semgrep - Patrones de seguridad" uses:
  semgrep/semgrep-action@v1
  with:
    config: >-
      p/python
      p/security-
      audit      p/owasp-
      top-ten
```

7.3 Fase Dependencies — SCA

Trivy escanea las dependencias de los 3 servicios:

```
- name: "Trivy - Escaneo de dependencias API" uses:
  aquasecurity/trivy-action@master
  with:
    scan-type: 'fs'
    scan-ref:   './api-gateway'
  severity: 'CRITICAL,HIGH'
  exit-code: '1' # Pipeline FALLA si encuentra vulnerabilidades
```

7.4 Fase Build — Escaneo de Imágenes

Se construyen las 3 imágenes Docker y se escanean con Trivy:

```
- name: "Build - API Gateway"
  run: docker build -t api-temp:${{ github.sha }} ./api-gateway

- name: "Trivy - Escaneo imagen API"
  uses: aquasecurity/trivy-action@master
  with:
    image-ref: 'api-temp:${{ github.sha }}' severity: 'CRITICAL'
    exit-code: '1' # Falla si hay CVE crítico
    ignore-unfixed: true
```

Fase Test — Pruebas Unitarias

Se ejecutan 23 pruebas con cobertura de código:

```
- name: "Ejecutar Pytest con Cobertura" run:
  |
  cd api-gateway
  pytest tests/ -v --tb=short --cov=app --cov-report=xml:coverage.xml
```

7.5 Tests implementados:

test_auth.py (5 tests): registro, duplicados, password corta, login, health check

test_workspaces.py (7 tests): CRUD, validación, IDOR

test_notes.py (11 tests): CRUD, filtros, etiquetas, pinned, IDOR

7.6 Fase DAST — OWASP ZAP

Se levanta la API y se escanea dinámicamente:

```
- name: "OWASP ZAP — Escaneo de API"
  uses: zaproxy/action-api-scan@v0.7.0
with:
  target: 'http://localhost:8000/openapi.json'
  fail_action: false
```

7.7 Fase Release — Docker Hub

Las imágenes se publican automáticamente con versionado semántico:

```
- name: "Build & Push — API Gateway RELEASE" uses:
  docker/build-push-action@v5
  with:
    context:      ./api-
    gateway push: true
    tags: |
      ${ env.DOCKER_HUB_USERNAME }/sevwork:api-${ steps.version.outputs.VERSION
    }}
      ${ env.DOCKER_HUB_USERNAME }/sevwork:api-latest
```

7.8 Escaneo de IaC — Checkov

Checkov valida los Dockerfiles y docker-compose:

```
- name: "Checkov — Escaneo de Dockerfiles y Compose" uses:
  bridgecrewio/checkov-action@v12
  with:
    directory: .
    framework: dockerfile,yaml
    skip_check: CKV_DOCKER_2,CKV_DOCKER_3,CKV2_DOCKER_1
```


8. RESULTADOS DE SEGURIDAD

8.1 Vulnerabilidades Encontradas y Remediadas

Dependencia	CVE	Severidad	Estado	Acción Tomada
python-jose	CVE-2024-33663	● Critical	✔ Resuelto	Migrado a PyJWT 2.12.0
python-jose	CVE-2024-33664	● Moderate	✔ Resuelto	Migrado a PyJWT 2.12.0
python-multipart	CVE-2024-53981	● High	✔ Resuelto	Actualizado a 0.0.27
python-multipart	CVE-2026-24486	● High	✔ Resuelto	Actualizado a 0.0.27

Ejemplo de Reporte Bandit

```
>> Issue: [B105:hardcoded_password_string] Possible hardcoded password
Severity: Low    Confidence: Medium
Location: app/config.py:15
```

Evaluación: Es el valor por defecto de JWT_SECRET_KEY en config.py. En producción se sobrescribe con variable de entorno. No es un riesgo real ya que pydantic-settings carga desde .env.

Ejemplo de Reporte Trivy (SCA)

```
api-gateway/requirements.txt (pip)
Total: 0 (HIGH: 0, CRITICAL: 0)
✔ Sin vulnerabilidades críticas o altas
```

Después de la remediación de python-jose y python-multipart, el escaneo de dependencias pasa limpio.

8.2 Controles de Seguridad Implementados

Control	Implementación	Evidencia
Contraseñas	bcrypt con salt automático (passlib)	deps.py:18
JWT	Access (30 min) + Refresh (7 días)	deps.py:34-47
IDOR	Filtro user_id en todas las queries	routers/notes.py:41, routers/workspaces.py:24
Rate Limiting	slowapi en API Gateway	main.py:23
Headers de Seguridad	HSTS, X-Frame-Options, CSP, X-XSS	main.py:54-63
Contenedores	Usuario no-root, imágenes slim	Dockerfile:15,29
Secretos	.env + GitHub Secrets + Gitleaks	.gitignore, devsecops.yml

8.3 Excepciones Documentadas

No hay excepciones de seguridad activas. Todas las vulnerabilidades conocidas han sido remediadas.

9. MONITOREO Y OBSERVABILIDAD

9.1 Observabilidad Implementada

Mecanismo	Herramienta	Comando
Estado de servicios	Docker Compose	docker compose ps
Logs en tiempo real	Docker Compose	docker compose logs -f [servicio]
Uso de recursos	Docker Stats	docker stats
Healthcheck PostgreSQL	pg_isready	Definido en docker-compose.yml
Healthcheck Redis	redis-cli ping	Definido en docker-compose.yml
Reportes de seguridad	GitHub Actions	Artefactos generados en cada ejecución del pipeline

Ejemplo — Salida de ‘docker compose ps’:

```
NAME                IMAGE                                STATUS      PORTS
>sw-frontend        robert0024/sevwork:fe-v1.0         Up (healthy) 0.0.0.0:3000-
>80/tcp
>sw-api              robert0024/sevwork:api-v1.0        Up (healthy) 0.0.0.0:8000-
>8000/tcp
>sw-worker           robert0024/sevwork:wk-v1.0         Up          -
>sw-postgres         postgres:15-alpine                 Up (healthy) 0.0.0.0:5432-
>5432/tcp
>sw-redis            redis:7-alpine                      Up (healthy) 0.0.0.0:6379-
```

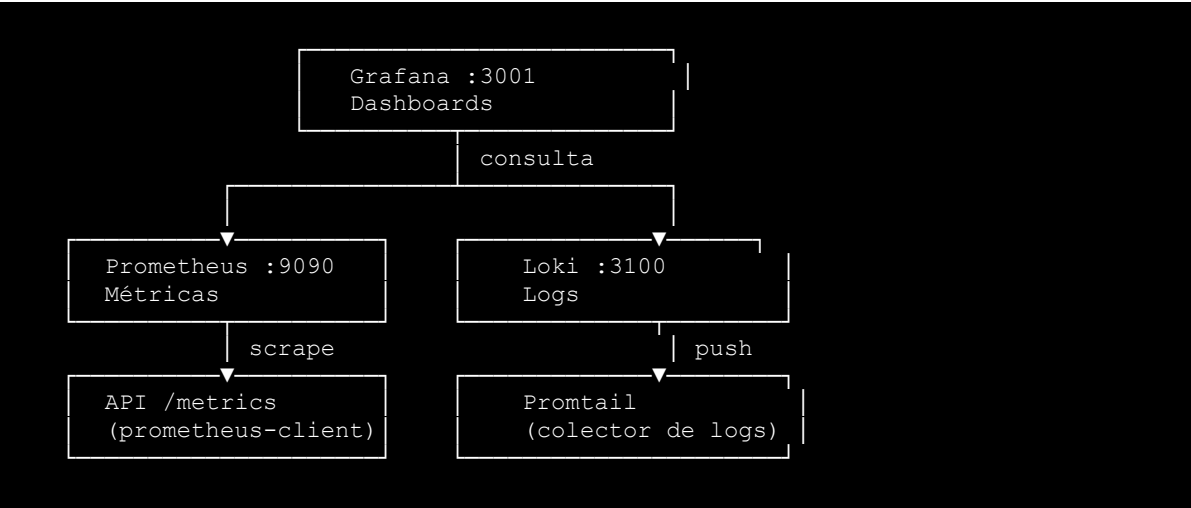
```
>6379/tcp
```

Healthcheck PostgreSQL en `docker-compose.yml`:

```
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d
${POSTGRES_DB}"]
  interval: 10s
  timeout: 5s
  retries: 5
  start_period: 30s
```

9.2 Arquitectura de Monitoreo Planificada

La carpeta monitoring/ está reservada para el stack completo de observabilidad:



Componente	Propósito	Estado
Prometheus	Recolección de métricas (latencia, requests, errores)	 Planificado
Grafana	Dashboards de visualización en tiempo real	 Planificado
Loki + Promtail	Centralización y consulta de logs de todos los servicios	 Planificado
Falco	Detección de comportamiento anómalo en runtime	 Planificado

9.3 Métricas Prioritarias Identificadas

Métrica	Descripción	Alerta sugerida
http_requests_total	Total de peticiones por endpoint y código HTTP	Error rate > 5%
http_request_duration_seconds	Latencia de la API por endpoint	P95 > 500ms
auth_login_failures_total	Intentos de login fallidos	> 10/min por IP
celery_tasks_total	Tareas procesadas por el Worker	Queue > 100 tareas
db_connections_active	Conexiones activas a PostgreSQL	> 80% del pool

10. CONCLUSIONES

10.1 Desafíos Encontrados

1. Compatibilidad de dependencias: La librería python-jose tenía CVEs críticos (CVE-2024-33663) sin parche disponible. La solución fue migrar completamente a PyJWT, lo que requirió adaptar la generación y verificación de tokens en todo el backend.
2. Falsos positivos en Bandit: Los valores por defecto en config.py son reportados como contraseñas hardcoded. Esto requirió documentar la justificación en lugar de eliminar los defaults, ya que son necesarios para el arranque local.
3. Configuración de OWASP ZAP en CI: ZAP requiere que la API esté levantada durante el escaneo, lo que añade complejidad al pipeline. Se resolvió levantando la API con uvicorn en background antes del escaneo.
4. Checkov y Docker: Algunos checks de Checkov (como HEALTHCHECK en Dockerfile) se excluyen intencionalmente porque los healthchecks se definen a nivel de docker-compose.yml, no en el Dockerfile.

10.2 Limitaciones

- No se implementó verificación de email en el registro (sin servidor SMTP)
- No hay funcionalidad de recuperación de contraseña
- El monitoreo con Prometheus/Grafana queda como trabajo futuro
- HTTPS no está configurado para desarrollo local (solo headers HSTS para producción)

10.3 Lecciones Aprendidas

1. La seguridad no se añade al final: Integrar herramientas de seguridad desde el inicio del pipeline evita acumulación de deuda técnica de seguridad.
2. El pipeline es tan fuerte como su política de fallo: Configurar exit-code: 1 en Trivy es lo que convierte un reporte informativo en una barrera real contra vulnerabilidades.
3. Documentar obstáculos es tan valioso como resolverlos: La migración de python-jose a PyJWT, documentada con CVEs específicos, demuestra gestión profesional de vulnerabilidades.
4. IaC permite reproducibilidad: Ansible y Terraform garantizan que el despliegue sea consistente en cualquier entorno.

10.4 Trabajo Futuro

- Implementar stack de monitoreo (Prometheus + Grafana + Loki)
- Añadir autenticación OAuth2 con proveedores externos (Google, GitHub)
- Implementar cifrado de notas en reposo
- Añadir pruebas de integración end-to-end con Playwright
- Configurar Falco para detección de anomalías en runtime
- Implementar CI/CD con despliegue a Kubernetes (K3s)